

MPI Parallelization of MLP Training on MNIST

Final Project Report

Allen Vinoy Aryan Patel

Spring 2026

GitHub Repository: github.com/KilroyHere/nn-training

1 Hypothesis

We investigate MPI acceleration of MLP training on MNIST through two families of techniques: data parallelism (flat DP, hierarchical DP, and Local-SGD) and model parallelism (sequential layer partitioning and pipelined variants). Our hypothesis is that communication structure, not only rank count, determines scaling: methods that reduce global synchronization pressure should achieve better epoch-time scaling while retaining comparable optimization behavior.

The results support this framing. In data parallelism, Local-SGD provides the best large-scale epoch-time reduction, flat DP remains the strongest fully synchronous baseline, and hierarchical DP does not amortize its extra phases at our current model size. In model parallelism, sequential partitioning remains communication-bound, while pipeline performance is highly sensitive to microbatch count and load balance.

2 Context

Member contributions.

- **Aryan Patel:** serial baseline, shared training utilities, and flat data-parallel baseline. hierarchical DP and local-SGD variants, launch scripts, and scaling experiments.
- **Allen Vinoy:** sequential model-parallel, pipeline parallel, load-balanced pipeline parallel experiments.

3 Prior Work

Distributed training has been studied across both data-parallel and model-parallel axes. [Sergeev and Del Balso \[2018\]](#) established synchronous data-parallel training via collective communication, which directly motivates our flat-DP and hierarchical-DP baselines. [Stich \[2019\]](#) showed that periodic weight synchronization (local SGD) reduces communication overhead while retaining convergence, motivating our K -step synchronization experiments. For model parallelism, [Narayanan et al. \[2019\]](#) demonstrated that pipeline-aware scheduling is essential and that bubble overhead is a first-order performance factor; our implementation adopts the same interleaved forward-backward schedule and uses the alpha-beta model to characterize when bubble reduction is overwhelmed by accumulated MPI startup latency.

4 Empirical Methodology

4.1 Problem Setup

We evaluate two distinct parallelization families on MNIST.

Data-parallel family (Serial, Flat DP, Hierarchical DP, Local SGD). These experiments use the main scaling model $784 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 10$, trained with SGD and cross-entropy under fixed optimizer settings and matched data splits. The execution modes are:

- **Serial:** single-process baseline.
- **Flat MPI Data Parallel (DP):** each rank computes local gradients and performs gradient all-reduce every step.
- **Hierarchical MPI DP:** per-step gradient synchronization is split into intra-node reduce, inter-node all-reduce (leaders), and intra-node broadcast.
- **Local SGD:** each rank performs local updates and averages model weights every K steps.

This family is used for the strong-scaling conclusions from $1n4r$ to $4n128r$.

Model-parallel family (MP and pipeline variants). These experiments use a deeper architecture with 8 weight matrices, $784 \rightarrow 512 \rightarrow 512 \rightarrow 256 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 10$, so layer partitioning across ranks is meaningful. Here we analyze sequential model parallelism and pipeline behavior under batch-size, microbatch-count, and world-size sweeps.

4.2 Sequential Baseline

The sequential baseline uses the same MLP, loss, and optimizer as all MPI variants. It runs a shuffled mini-batch loop over training data and reports `epoch_time_ms` as *training-loop-only* wall time (validation evaluation is measured but excluded from epoch timing). Validation is excluded from epoch timing in all backends, so comparisons reflect training cost only.

4.3 Parallel Design

4.3.1 Design Overview

Serial training has no communication. Data-parallel methods synchronize gradients or weights with collectives. Model parallelism partitions layers and exchanges intermediate tensors point-to-point. For clarity, we therefore present separate algorithms for (1) serial training, (2) data-parallel training, and (3) model-parallel training.

4.3.2 Serial Design

Algorithm 1 Serial training epoch

Require: shuffled indices I , batch size B , parameters θ

Ensure: updated θ after one epoch

- 1: **for** $t = 0, B, 2B, \dots, |I| - B$ **do**
 - 2: gather batch (x, y) from I
 - 3: $g \leftarrow \nabla \ell(\theta; x, y)$
 - 4: $\theta \leftarrow \theta - \eta g$
 - 5: **end for**
-

Serial. Single-process SGD over shuffled mini-batches; no MPI calls.

4.3.3 Data-Parallel Designs

Algorithm 2 Data-parallel epoch (flat DP / hierarchical DP / local SGD)

Require: shuffled index list I , global batch size B , rank count P

Ensure: updated model parameters after one epoch

```

1: for global batch offset  $t = 0, B, 2B, \dots, |I| - B$  do
2:   gather local mini-batch  $(x_r, y_r)$  from  $I$ 
3:   compute local gradients  $g_r \leftarrow \nabla \ell(\theta; x_r, y_r)$ 
4:   if mode is Flat DP then
5:      $g \leftarrow \text{AllreduceMean}(g_r)$ 
6:   else if mode is Hierarchical DP then
7:      $g \leftarrow \text{IntraReduce} \rightarrow \text{InterAllreduce} \rightarrow \text{IntraBcast}$ 
8:   else if mode is Local SGD then
9:     set  $g \leftarrow g_r$ ; synchronize weights only every  $K$  steps
10:  end if
11:  apply SGD update with synchronized quantity (gradient or weight)
12: end for

```

Flat DP. Each rank processes a disjoint shard of each global data batch, computes local gradients, then uses `MPI_Allreduce` on layer gradients to form the global mean gradient before applying the update.

Hierarchical DP. Gradient synchronization is decomposed into three phases: `MPI_Reduce` within a node, `MPI_Allreduce` across node leaders, then `MPI_Bcast` within each node. This reduces inter-node participants but introduces extra synchronization phases.

Local SGD. Ranks execute local updates independently and synchronize model weights via one fused `MPI_Allreduce` every K steps ($K \in \{10, 50\}$ in this report). This trades statistical accuracy for lower communication frequency.

4.3.4 Model-Parallel and Pipeline Designs

Algorithm 3 Model-Parallel batch step (Sequential / Pipeline / Load-Balanced)

Require: ranks P , rank r , batch (x, y) , microbatches M (if pipelined)

```

1: if mode is Load-Balanced then assign  $\theta_r$  by layer cost else assign naively
2: if mode is Model Parallel then
3:   receive  $a_{\text{in}}$  (or  $x$ ), compute forward  $a_{\text{out}}$ , send to  $r + 1$ 
4:   receive  $g_{\text{out}}$  (or  $\nabla \ell$ ), compute backward  $\nabla \theta_r$  and  $g_{\text{in}}$ , send  $g_{\text{in}}$  to  $r - 1$ 
5: else if mode  $\in \{\text{Pipeline, Load-Balanced}\}$  then
6:   split  $(x, y)$  into  $M$  microbatches; initialize  $\nabla \theta_r^{\text{accum}} \leftarrow 0$ 
7:   for step  $t = 0, \dots, M + P - 2$  do
8:     if  $t \in [0, M - 1]$  then compute forward microbatch  $t$ , send  $a_{\text{out}}$ 
9:     if  $t - (P - 1) \in [0, M - 1]$  then compute backward microbatch, add to  $\nabla \theta_r^{\text{accum}}$ 
10:  end for
11:   $\nabla \theta_r \leftarrow \nabla \theta_r^{\text{accum}} / M$ 
12: end if
13: apply SGD update to  $\theta_r$  using  $\nabla \theta_r$ 

```

Model Parallel and Pipelining. Layers are partitioned across ranks. Forward activations and backward gradients are exchanged via point-to-point MPI. We evaluate three partitioning and scheduling variants:

- **Model Parallel:** A non-pipelined schedule where a single global batch is passed sequentially across ranks, leaving $(P - 1)$ ranks idle at any given time.
- **Pipeline Parallel:** The global batch is split into M microbatches. Ranks execute a staggered schedule, interleaving forward and backward passes (offset by $P - 1$ steps) to fill the pipeline. Gradients are accumulated and applied synchronously at the end.
- **Load-Balanced Pipeline:** Instead of a naive L/P layer division, layers are assigned greedily based on computational cost (using $d_{\text{in}} \times d_{\text{out}}$ parameter counts) to equalize the workload per rank and minimize pipeline bottleneck stages.

4.4 Communication Cost Model (Alpha-Beta)

We use the standard latency-bandwidth model:

$$T_{\text{msg}}(m) = \alpha + \beta m, \quad (1)$$

where α is startup latency, β is per-byte (or per-float) transfer cost, and m is message size.

Table 1: Alpha-Beta communication model summary by method.

Method	Per-step or per-batch approximation
Flat DP	$T_{\text{flat}}(m_g) \approx 2(P - 1)\alpha + 2\frac{P-1}{P}\beta m_g$. Scales well at moderate P , but latency grows with rank count.
Hierarchical DP	$T_{\text{hier}}(m_g) \approx (2 \log q \alpha + 2\frac{q-1}{q}\beta m_g) + (2(N - 1)\alpha + 2\frac{N-1}{N}\beta m_g)$, with $P = Nq$. Reduces inter-node participants, but adds extra sync phases.
Local SGD	$T_{\text{local,step}} \approx T_{\text{compute}} + \frac{1}{K}T_{\text{sync}}(m_w)$. Larger K lowers communication cost but can weaken statistical freshness.
Model Parallel	$T_{\text{mp,step}} \approx \sum_{r=1}^P (C_r + (\alpha + \beta m_a) + (\alpha + \beta m_b))$. Stage imbalance and boundary traffic dominate runtime.
Pipeline Parallel	$T_{\text{batch}} = (M + L - 1) \max(\frac{C_0}{M}, \alpha + \frac{\beta m_0}{M}) + (L - 1) \min(\frac{C_0}{M}, \alpha + \frac{\beta m_0}{M})$, $\text{bubble}(L, M) = \frac{L-1}{M+L-1}$. More microbatches reduce bubbles, but too many can amplify latency cost.

4.5 Implementation Details

The project is implemented in C++17 with MPI and BLAS-backed dense linear algebra. To keep comparisons fair, serial and distributed backends share the same model definition, optimizer semantics, data preprocessing, and epoch-timing convention.

Shared runtime and kernels. All backends use BLAS-backed primitives for dense operations (`sgemm/saxpy/sgemv`) instead of scalar loops. This reduces per-step compute overhead and keeps arithmetic kernels consistent across experiments.

Backend-specific execution paths. The codebase isolates execution strategies into dedicated binaries and launch scripts for serial, flat DP, hierarchical DP, local-SGD, and model parallelism. This avoids mode-specific branching in hot loops and keeps experimental control explicit at launch time.

Performance-focused optimizations. Training hot loops use buffer reuse and locality-aware data movement: (i) preallocated mini-batch feature/label buffers, (ii) contiguous gather writes, and (iii) reusable gradient/workspace buffers in the shared MLP path. Release builds use `-O3`.

Fair-comparison controls. The serial path is intentionally tuned with the same kernel and build strategy as parallel runs, so reported speedups are not inflated by avoidable single-rank overhead. Since model parallelism uses a separate forward/backward path, its performance depends less on the BLAS and buffer optimizations above, and more on how evenly layers are distributed across ranks and how quickly activations and gradients can be passed between neighboring ranks.

4.6 Experimental Setup

All experiments were launched with Slurm `srun`. To avoid thread-level oversubscription, we pinned `OMP_NUM_THREADS=1`, `OPENBLAS_NUM_THREADS=1`, and `MKL_NUM_THREADS=1`. Unless noted otherwise, runs use 10 epochs and seed = 42.

4.6.1 Data-Parallel Setup

Goal: measure strong scaling across rank count for serial/DP-family methods.

Fixed configuration:

- model: $784 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 10$
- global batch: 1024
- train/validation samples: 50,000/10,000
- learning rate: 0.03

Sweep axis:

- rank ladder (notation: n =nodes, r =MPI ranks): $1n4r \rightarrow 1n16r \rightarrow 1n32r \rightarrow 2n64r \rightarrow 4n128r$

4.6.2 Model-Parallel / Pipeline Setup

Goal: characterize partitioning and communication/scheduling effects in MP and pipeline runs.

Fixed configuration:

- model: $784 \rightarrow 512 \rightarrow 512 \rightarrow 256 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 10$
- pipeline default: load-balanced layer assignment (`-load-balance-layers`)

Sweep axes:

- **Batch-size sweep:** batch = 256...4096, fixed world size = 8 with 4×2 layout (4 nodes, 2 tasks per node), microbatch count $M = 32$.
- **Microbatch sweep:** fixed batch = 4096, world size = 8 with 2×4 layout, varying microbatch count $M \in \{8, 32, 64\}$.
- **World-size sweep:** fixed batch = 4096, $M = 32$, world size $ws \in \{2, 4, 6, 8\}$ with 2 tasks per node (1–4 nodes). Both unbalanced and load-balanced pipeline variants are run to quantify the partitioning benefit.

4.7 Results

DP and MP results are reported as two method families with different workload goals and model shapes (Section 4.6); comparisons are therefore interpreted within family (DP vs DP, MP vs MP), not as direct absolute DP-vs-MP rankings.

4.7.1 Data-Parallel Results

Epoch time vs. cluster size. Figure 1 shows epoch time vs cluster size with serial baseline and ideal scaling reference. Local-SGD achieves the best time at 128 ranks ($K=50$ fastest), while flat DP remains more efficient than hierarchical DP for this model size and workload regime. A likely explanation is that `MPI_Allreduce` in the flat-DP path benefits from mature, topology-aware collective implementations in the system MPI stack, whereas the explicit two-level hierarchical sequence (intra-reduce, inter-allreduce, intra-bcast) adds extra synchronization phases that are not sufficiently amortized at the current gradient payload.

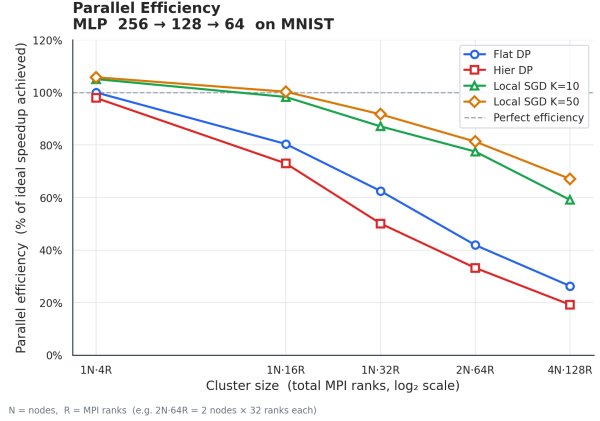
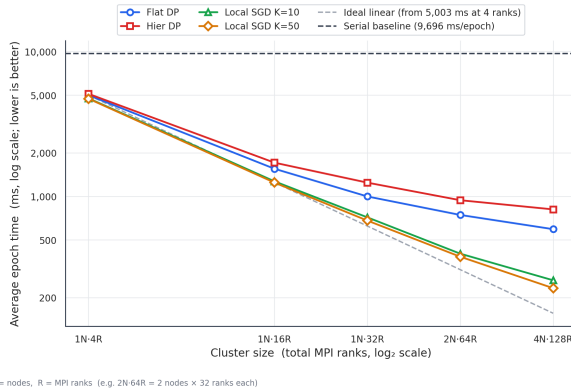


Figure 1: Epoch time vs cluster size with serial baseline and ideal line (MLP 256 $\rightarrow$ 128 $\rightarrow$ 64, across synchronization methods. MNIST, 10 epochs).

Strong-scaling speedup and efficiency. Using flat DP at 1n4r as baseline (≈ 4143 ms/epoch), strong-scaling speedup and efficiency are:

Table 2: Strong scaling for flat DP (baseline: 1n4r).

Config	Time (ms)	Speedup $S(p)$	Efficiency $E(p)$
1n4r	4143	1.00 $\times$	100.0%
1n16r	1125	3.68 $\times$	92.1%
1n32r	652	6.35 $\times$	79.4%
2n64r	408	10.16 $\times$	63.5%
4n128r	292	14.21 $\times$	44.4%

Figure 2 summarizes DP efficiency across methods: flat DP, hierarchical DP, and Local-SGD ($K = 10, 50$). Efficiency decays with cluster size as communication overhead becomes a larger fraction of step time. Compared to flat DP, hierarchical DP trails at all tested scales in the current runs, while Local-SGD surpasses flat DP speedup as K increases, reaching 27.39 $\times$ speedup at 4n128r ($K=50$) relative to the flat-DP 4-rank baseline.

Communication breakdown and synchronization cost. Timing breakdowns from logs show:

- flat DP: communication cost appears primarily in `allreduce_ms`, while `other_ms` remains non-negligible at high ranks. Combined with better end-to-end epoch times, this is consistent with (but does not by itself prove) topology-aware optimization of `MPI_Allreduce` by the MPI runtime; the exact per-call internal collective algorithm is not fully exposed in public documentation for the production MPI stack used on this system [NERSC, 2026a,b, HPE Cray Programming Environment, 2025].
- hierarchical DP: cost is split into `intra_reduce_ms`, `inter_allreduce_ms`, and `intra_bcast_ms`, confirming additional synchronization phases.
- local-SGD: `sync_ms` is much smaller than per-step DP comm cost because communication occurs only every K steps.

Accuracy trade-off is modest for the fastest local-SGD settings in these runs: at 128 ranks, $K=50$ is fastest with a small validation-accuracy drop relative to flat DP.

Local-SGD synchronization sensitivity (K). Across the tested Local-SGD settings ($K \in \{10, 50\}$), increasing K reduces synchronization overhead and improves epoch time in large-scale runs, with only a modest validation-accuracy change in the current workload regime.

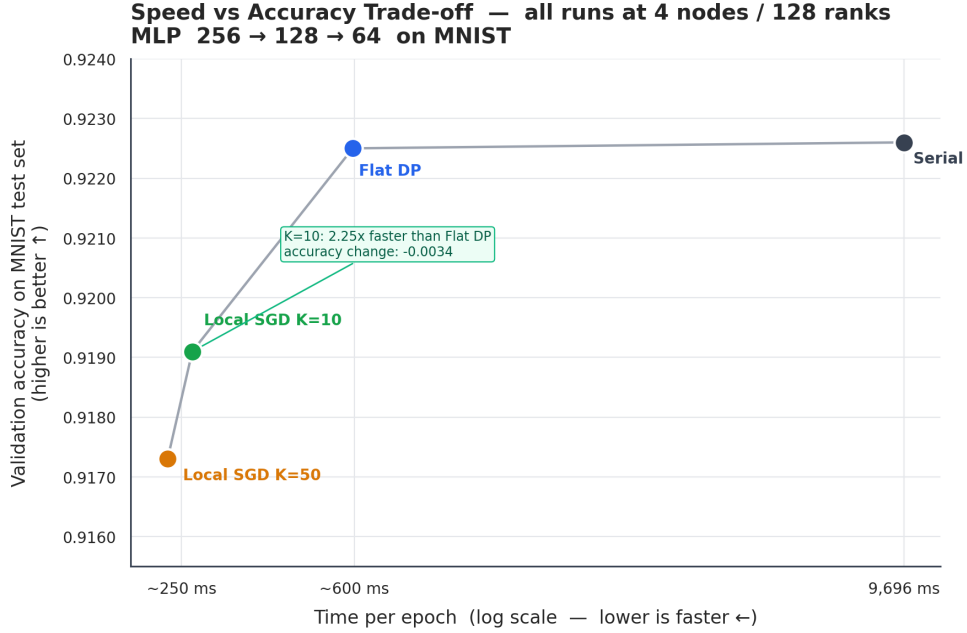


Figure 3: Speed-vs-accuracy tradeoff at $4n128r$ for serial, flat DP, and Local-SGD ($K = 10, 50$).

DP takeaway. Within the DP family, communication structure is the dominant performance factor at scale: Local-SGD gives the strongest epoch-time reduction in current runs, flat DP remains the best fully synchronous baseline, and hierarchical DP adds synchronization phases that are not sufficiently amortized at this model size.

4.7.2 Model-Parallel Results

Serial vs. model parallel vs. pipeline (batch size sweep). Figure 4 shows epoch time as batch size varies from 256 to 4096 at $ws = 8$. Sequential model parallel runs at 72–79 s/epoch regardless of batch size — roughly $2\times$ the serial baseline of 36–39 s — because the non-pipelined schedule serializes all ranks: total step time equals the *sum* of every rank’s compute plus communication. Pipeline parallel is strongly batch-sensitive: at batch = 256 the microbatch size is only 8 rows, making per-message latency the dominant cost (≈ 140 s/epoch). As batch size grows and microbatches become larger, pipeline time falls sharply, approaching the model-parallel line at batch = 2048 and crossing below it at batch = 4096 when microbatch size is large enough to amortize startup latency.

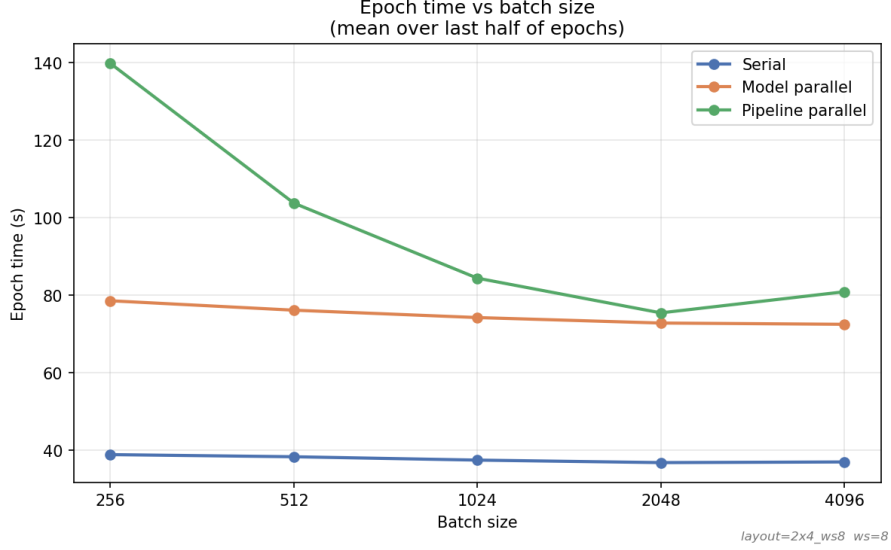
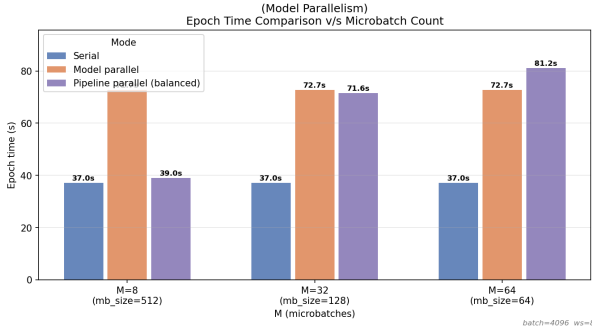
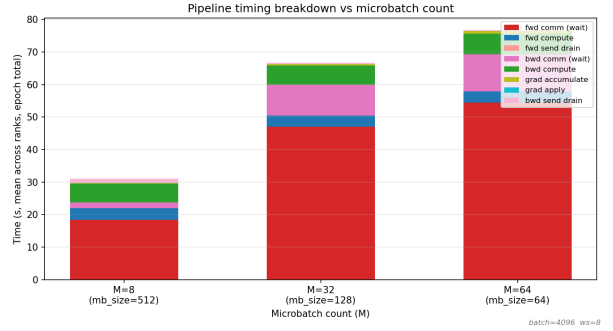


Figure 4: Epoch time vs. batch size ($ws = 8$, layout 4×2 , $M = 32$ microbatches).

Microbatch sensitivity and pipeline idle time. Figure 5 fixes batch = 4096 and varies microbatch count M . The result is counterintuitive: *fewer microbatches are faster*.



(a) Epoch time vs. M .



(b) Per-category idle time breakdown.

Figure 5: Microbatch sensitivity and pipeline idle time (batch = 4096, $ws = 8$).

At $M = 8$ (microbatch size 512 rows) pipeline epoch time is ≈ 39 s, nearly matching serial (≈ 37 s). At $M = 32$ it rises to ≈ 71.6 s, and at $M = 64$ to ≈ 81.2 s, despite $M = 64$ having the best theoretical pipeline efficiency (90.1%). The timing breakdown (Figure 5b) reveals why: forward communication wait (`fwd_comm`) — the time each rank sits idle waiting for the next activation — grows from ≈ 18 s at $M = 8$ to ≈ 55 s at $M = 64$, scaling roughly linearly with M . Each of the M microbatches incurs a separate MPI send latency α regardless of message size; for small MNIST activations α dominates over bandwidth cost βm , so total idle time accumulates as $M\alpha$ and overwhelms the theoretical bubble reduction.

Pipeline vs. load-balanced pipeline. Figure 6a shows epoch time across world sizes $ws \in \{2, 4, 6, 8\}$ at batch = 4096 and $M = 32$. Model Parallel is flat at ≈ 72 s regardless of world size, as expected. Pipeline improves modestly with world size; the load-balanced variant reduces epoch time by 2–6% over naive round-robin assignment at $ws = 2, 4, 6$ (Figure 6b). The improvement peaks at $ws = 6$ (+6.1%), where naive assignment places the two heaviest layers on the earliest ranks; the

balanced partitioner redistributes by cost. At $ws = 8$ both strategies assign exactly one layer per rank, so the result is identical (+0.0%). Across all configurations, serial (≈ 37 s) remains faster than every parallel variant.

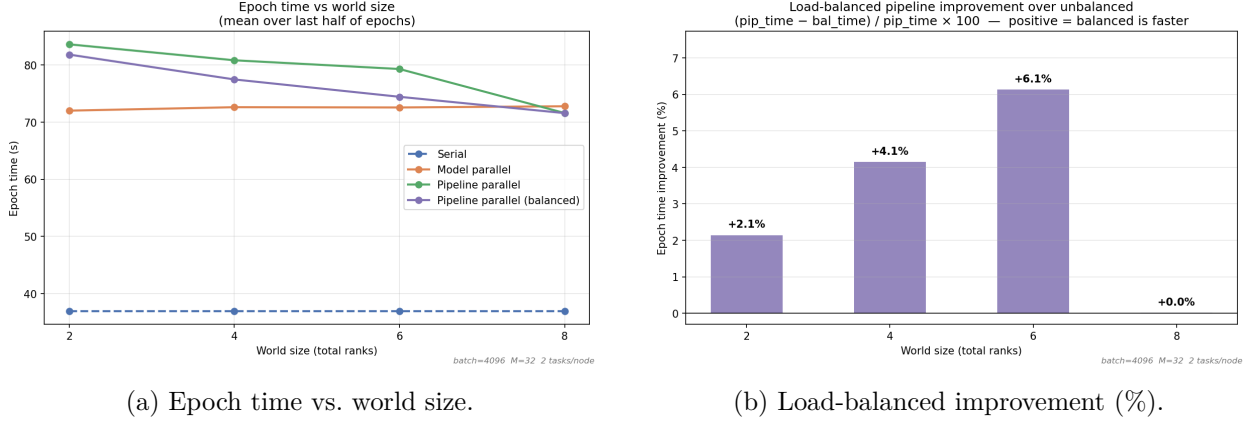


Figure 6: World-size scaling and load-balance effect (batch = 4096, $M = 32$, 2 tasks/node).

5 Challenges and Obstacles

Challenge 1: Microbatch count unexpectedly dominated pipeline performance. The bubble fraction $\frac{L-1}{M+L-1}$ decreases as M grows, predicting that more microbatches should reduce pipeline idle time and improve throughput. Instead, $M = 8$ nearly matched serial (≈ 39 s) while $M = 64$ was the slowest (≈ 81 s) despite having the highest bubble fraction of 47% vs 9.9% respectively. The timing breakdown revealed that forward communication wait (`fwd_comm`) grows roughly as $M\alpha$: each microbatch incurs a fixed MPI startup cost α regardless of message size, and for small MNIST activations this latency dominates over bandwidth cost βm . The bubble model is therefore incomplete — it only captures compute idleness and ignores accumulated communication latency, which becomes the binding constraint at large M .

Challenge 2: Flat MPI_Allreduce outperformed our hierarchical design at scale. We expected hierarchical DP to win at higher rank counts by reducing inter-node participants, but on Perlmutter the flat DP path was consistently faster in end-to-end epoch time. A key obstacle was that this contradicted our initial communication-structure intuition and forced us to revisit system-level effects. Our working explanation is that the production MPI stack on Perlmutter provides highly optimized, topology-aware collective algorithms for MPI_Allreduce, while our explicit two-level hierarchy (`Reduce` $\rightarrow$ leader `Allreduce` $\rightarrow$ `Bcast`) introduces extra synchronization phases and software overhead that were not amortized at our model size.

Challenge 3: Keeping cross-backend comparisons fair required strict measurement discipline. Serial, data-parallel, and model-parallel paths use different communication patterns and partially different kernels, so naive timing comparisons were easy to misinterpret. We had to enforce matched launch semantics, a shared epoch-time definition, and an optimized serial baseline (BLAS-backed kernels, reuse-oriented buffers, release builds) before interpreting speedups. This added engineering overhead but was necessary to attribute differences to algorithmic choices rather than mismatched implementation quality.

References

- HPE Cray Programming Environment. intro_mpi documentation. https://cpe.ext.hpe.com/docs/25.03/mpt/mpich/intro_mpi.html, 2025. Accessed: 2026-05-12.
- Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil Devanur, Gregory Ganger, Phillip Gibbons, and Matei Zaharia. PipeDream: Fast and efficient pipeline parallel DNN training. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, 2019.
- NERSC. Cray mpich. <https://docs.nersc.gov/development/programming-models/mpi/cray-mpich>, 2026a. Accessed: 2026-05-12.
- NERSC. Network (perlmutter slingshot interconnect). <https://docs.nersc.gov/performance/network>, 2026b. Accessed: 2026-05-12.
- Alexander Sergeev and Mike Del Balso. Horovod: Fast and easy distributed deep learning in TensorFlow. In *Proceedings of the 31st Conference on Neural Information Processing Systems Workshop*, 2018.
- Sebastian U. Stich. Local SGD converges fast and communicates little. In *International Conference on Learning Representations*, 2019.